STUDY GUIDE

QRisk Scanner

How a quantum-risk crypto scanner works, from the big idea down to every module, protocol message and test.

5
scanning layers: TLS, files, SSH, code, config

4
ratings, from Critical to Quantum-safe

6
outputs, including a standard CBOM and an Arabic PDF

97
tests, including a live Docker lab

QRisk Scanner 0.1.0 · Python 3.10+ · guide written November 2026
Read Part 1 for the big picture (no code needed). Read Part 2 alongside the source code.

Contents

Part 1 · The big picture

1. The problem: quantum computers vs today's crypto
2. What QRisk does, in one picture
3. The four ratings
4. NIST deadlines
5. The Quantum Readiness score
6. Using QRisk
7. Reading real results: the lab

Part 2 · How it works inside

8. Map of the code
9. The data model and the life of a finding
10. The rules engine
11. Layer 1: TLS servers
12. Layer 2: certificate and key files
13. Layer 3: SSH servers
14. Layer 4: source code
15. Layer 5: configuration files
16. Outputs: terminal, JSON, CBOM, HTML, PDF, Arabic
17. Robustness and security by design
18. Testing and the Docker lab

Reference

19. Glossary
20. A study plan with exercises

PART 1

The big picture

What problem QRisk solves, what it looks at, how it judges what it finds, and how to read its results. No programming knowledge is needed for this part.

1. The problem: quantum computers vs today's crypto

Almost every secure connection on the internet (HTTPS, SSH, VPNs) uses two kinds of cryptography:

- **Public-key cryptography** (RSA, elliptic curves such as ECDSA, X25519 and P-256, and Diffie-Hellman). Two parties use it to *agree on a secret key* over an open network and to *prove who they are* with certificates and signatures.
- **Symmetric cryptography** (AES, ChaCha20) and **hashes** (SHA-256). These encrypt the actual data with that secret key and check that nothing was changed.

A large enough quantum computer changes this in two ways:

ALGORITHM	ATTACK	EFFECT
Shor's algorithm	Factors numbers and solves discrete logarithms fast	**Breaks** RSA, ECC and Diffie-Hellman completely, at any key size
Grover's algorithm	Speeds up brute-force search	Roughly **halves** symmetric strength: AES-128 behaves like ~64-bit, AES-256 like ~128-bit, which is still fine

So the real danger is **public-key crypto**. And the danger exists *today*, before such a computer is built, because of the attack below.

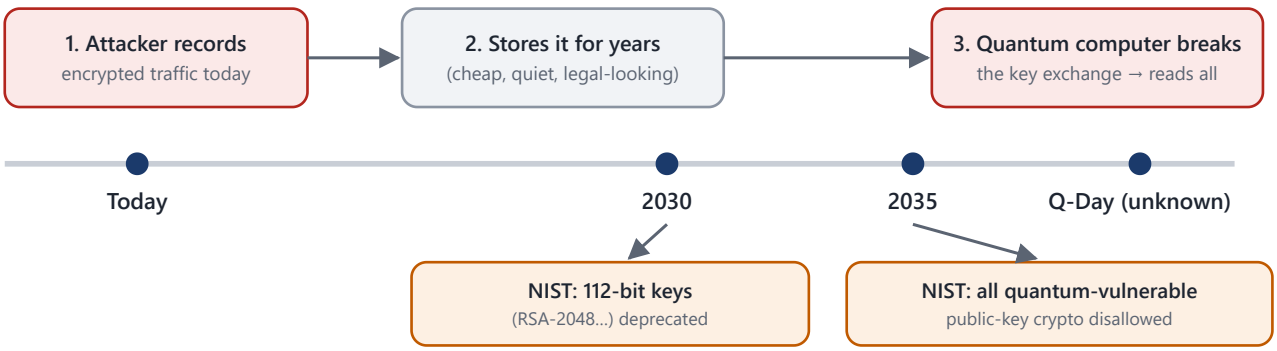


Figure 1 · "Harvest now, decrypt later": traffic recorded today can be decrypted once a quantum computer exists. Data that must stay secret for years is already at risk.

THE FIX THE WORLD IS MOVING TO

NIST has standardised new **post-quantum** algorithms: **ML-KEM** (FIPS 203) for agreeing on keys, and **ML-DSA** (FIPS 204) and **SLH-DSA** (FIPS 205) for signatures. Today they are usually deployed as **hybrids** that combine a classical and a post-quantum algorithm, for example `X25519MLKEM768`. If either half holds, the connection stays safe.

Before an organisation can migrate, it must answer a simple question that turns out to be hard: **where do we use cryptography, and which of it is vulnerable?** That inventory is what QRisk produces.

2. What QRisk does, in one picture

QRisk is a command-line tool. You point it at things to scan, and it produces a rated inventory of the cryptography it finds.

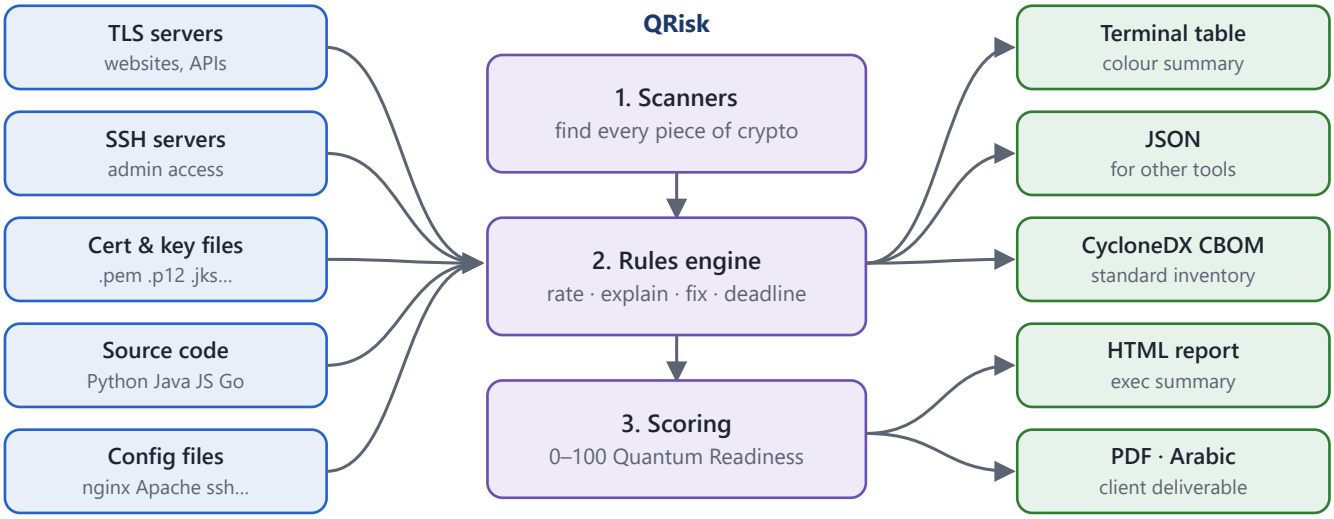


Figure 2 · Five kinds of input, three core stages, five kinds of output.

1. **Scanners** find cryptography. Each of the five *layers* knows how to look at one kind of thing: a live TLS server, a live SSH server, files on disk, source code, or server configuration.
2. The **rules engine** judges every piece it finds. It gives each one a *rating*, a plain-English *reason*, a concrete *fix*, and the *NIST deadline* that applies.
3. **Scoring** summarises everything as a 0–100 **Quantum Readiness** score with a grade.

Every piece of cryptography found is called a **finding**. A finding always says *what* was found (the asset, e.g. `RSA-2048 certificate`), *where* (a host and port, or a file and line), and the verdict.

LAYER	LOOKS AT	EXAMPLE FINDINGS
1 · TLS	Live HTTPS/TLS servers	TLS 1.0 enabled · `X25519MLKEM768` offered · RSA-1024 certificate
2 · Files	Certificates and keys on disk	Expired certificate · ML-DSA-65 key · JKS keystore
3 · SSH	Live SSH servers	`ssh-rsa` host key · `mlkem768x25519-sha256` · RSA-3072 host key
4 · Code	Python, Java/Kotlin, JS/TS, Go	`hashlib.md5()` at `app.py:6` · AES in ECB mode · `verify=False`
5 · Config	nginx, Apache, OpenSSL, OpenSSH	`ssl_protocols TLSv1` · `@SECLEVEL=1` · no post-quantum group configured

WHY FIVE LAYERS?

A live scan tells you what a server *actually does*. Config and code scans tell you what someone *asked for*. These can differ: the lab's weak server is configured to offer 3DES, but its OpenSSL silently drops it. A real audit needs both views, and the differences are findings in their own right.

3. The four ratings

Every finding gets exactly one of four ratings. They answer two questions: *is it weak today?* and *will a quantum computer break it?*

RATING	MEANING	EXAMPLES	POINTS
Critical	Weak even today. Fix first.	RSA-1024, SHA-1 signatures, TLS 1.0/1.1, RC4, 3DES, static-RSA key exchange, `ssh-rsa`, `hmac-md5`, `verify=False`	0
Quantum-vulnerable	Fine today, broken by a quantum computer	RSA-2048/3072, ECDSA, Ed25519, ECDHE, X25519, Diffie-Hellman	25
Hybrid	Post-quantum + classical combined: the recommended transition	`X25519MLKEM768`, `SecP384r1MLKEM1024`, `mlkem768x25519-sha256`	100
Quantum-safe	Post-quantum standard, or symmetric crypto strong enough	ML-KEM, ML-DSA, SLH-DSA, AES-256-GCM, ChaCha20-Poly1305	100

WHY DOES HYBRID SCORE THE SAME AS PURE POST-QUANTUM?

Hybrid is what browsers, Cloudflare and OpenSSH actually deploy today, and it's *safer* during the transition: if a flaw is ever found in the new algorithm, the classical half still protects you. Scoring it lower would push people toward the riskier choice.

Every verdict comes with four things

```
Asset:          RSA-2048 certificate (shop.example.com)
Rating:         Quantum-vulnerable
Reason:         RSA-2048 is safe today but Shor's algorithm breaks RSA on a quantum computer.
Recommendation: Add to your migration plan for ML-DSA (FIPS 204) certificates once your CA and
                clients support them. Keep certificate lifetimes short so you can rotate quickly.
NIST:           NIST IR 8547: deprecated after 2030, disallowed after 2035
```

Recommendations are written in the language of the thing being fixed. A config finding for nginx says `ssl_protocols TLSv1.2 TLSv1.3;` ; the same problem in Apache says `SSLProtocol -all +TLSv1.2 +TLSv1.3` .

4. NIST deadlines

NIST IR 8547 sets the timeline for retiring quantum-vulnerable public-key algorithms. QRisk attaches the matching deadline to each finding:

GROUP	EXAMPLES	DEADLINE
Already weak	RSA < 2048, SHA-1 signatures, MD5, DSA, TLS 1.0/1.1	Already disallowed
112-bit security	RSA-2048, P-224, ffdhe2048, DH group 14	Deprecated after **2030**, disallowed after **2035**
128-bit security and above	RSA-3072+, P-256, P-384, X25519, Ed25519	Disallowed after **2035**
Post-quantum	ML-KEM, ML-DSA, SLH-DSA	Approved (FIPS 203 / 204 / 205)

Rule enforced by a test: a Critical finding never carries a *future* deadline. Critical means broken now. (An early version labelled static RSA “deprecated after 2030”; code review caught it.)

5. The Quantum Readiness score

The score condenses hundreds of findings into one number a manager can track. It is calculated in four steps:

1. **Points per finding:** Quantum-safe 100, Hybrid 100, Quantum-vulnerable 25, Critical 0.
2. **Only public-key categories measure readiness:** key exchange, certificate keys and signatures, host keys, other key pairs and signatures. Hygiene categories (protocol version, symmetric ciphers, MACs, hashes, TLS settings) count *only when they are Critical*.
3. **Average per category, then per target, then across targets.** So 15 cipher suites can’t outweigh one certificate, and every scanned target counts equally.
4. **Any Critical finding caps the score at 49.** Nothing is “quantum-ready” while something is broken today.

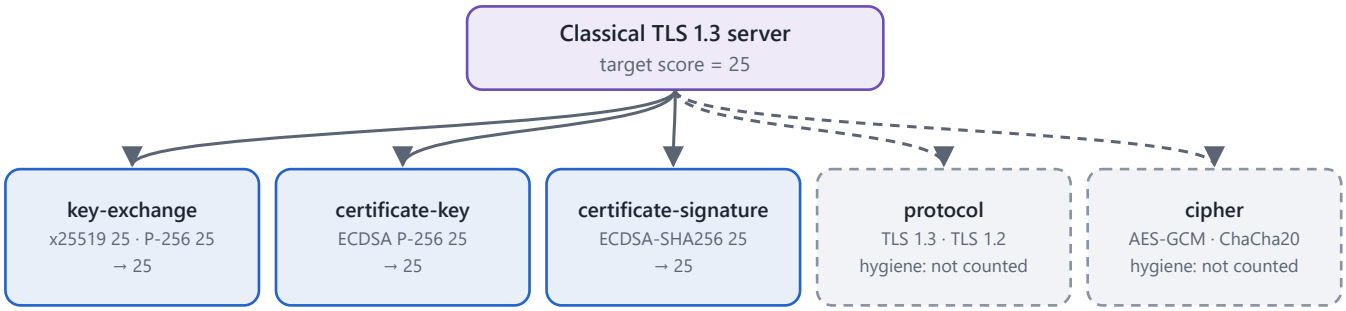


Figure 3 · Worked example: a well-configured but classical TLS 1.3 server scores 25. AES-GCM and TLS 1.3 are sound, but they don't protect against a quantum computer, so they don't raise the score.

SCORE	GRADE	TYPICAL SITUATION
80–100	Quantum-ready	Post-quantum or hybrid key exchange *and* no classical public keys left
50–79	In transition	Hybrid key exchange deployed, certificates or host keys still classical
25–49	Quantum-vulnerable	Classical crypto that is sound today (or something critical capped the score)
0–24	At risk	Critical problems that need fixing now

WHERE IN THE CODE

`src/qrisk/scoring.py` (69 lines). `QUANTUM_CATEGORIES` lists the public-key categories; `readiness_score()` does steps 2–4.

6. Using QRisk

```
qrisk scan --tls example.com                                    # one HTTPS server
qrisk scan --tls host:8443 --ssh github.com --path ./certs    # mix any layers
qrisk scan --code ./src                                         # find crypto in source code
qrisk scan --config /etc/nginx --config /etc/ssh                # audit server configs
qrisk scan --tls example.com --html r.html --pdf r.pdf --cbom c.json --json r.json
qrisk scan --tls example.com --pdf report-ar.pdf --lang ar   # Arabic, right-to-left
qrisk scan --path /etc/ssl --fail-under 50                      # CI gate: exit 2 if score < 50
```

OPTION	WHAT IT DOES
`--tls` / `--ssh` / `--path` / `--code` / `--config`	Targets for layers 1–5. Each can be repeated.
`--json` / `--cbom` / `--html` / `--pdf`	Write results in that format (the terminal table is always printed unless `-q`).
`--lang ar`	Arabic HTML/PDF report.
`--fail-under N`	For CI: fail if the score is below N, if any target couldn't be scanned, or if nothing was found.
`--allow-empty`	With `--fail-under`: finding nothing is acceptable (errors still fail).
`--config-follow-external`	Let config scans follow includes and certificate paths outside the scanned folder.
`--timeout` / `-q`	Network timeout in seconds (default 5) / don't print the terminal table.

7. Reading real results: the lab

The project includes a Docker lab (section 18) with deliberately weak, classical and post-quantum servers. Here is what a full five-layer scan of it finds, using the current scoring rules:

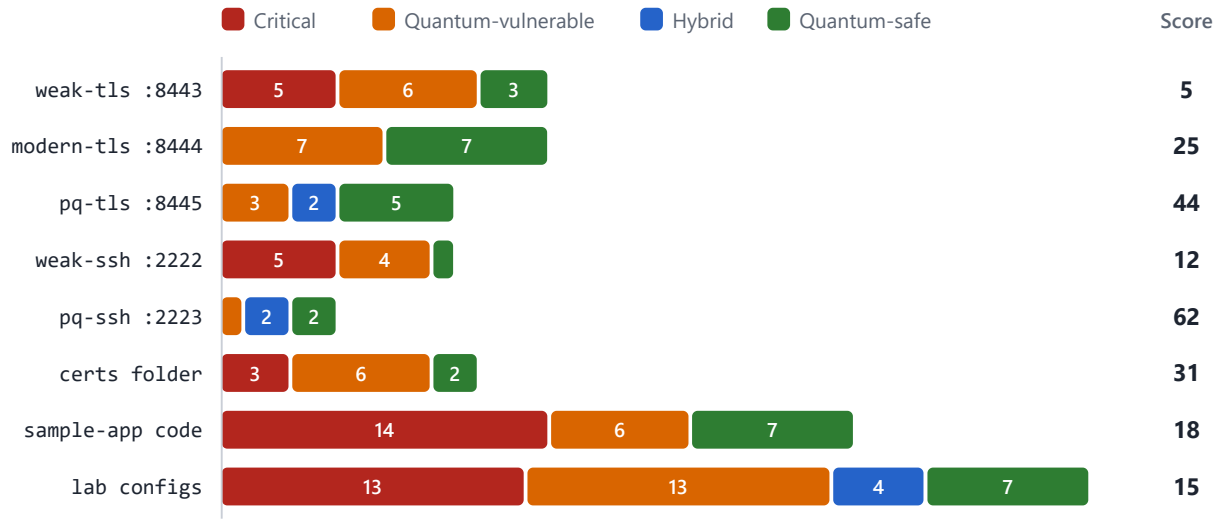


Figure 4 · Findings per lab target, coloured by rating, with each target's readiness score on the right.

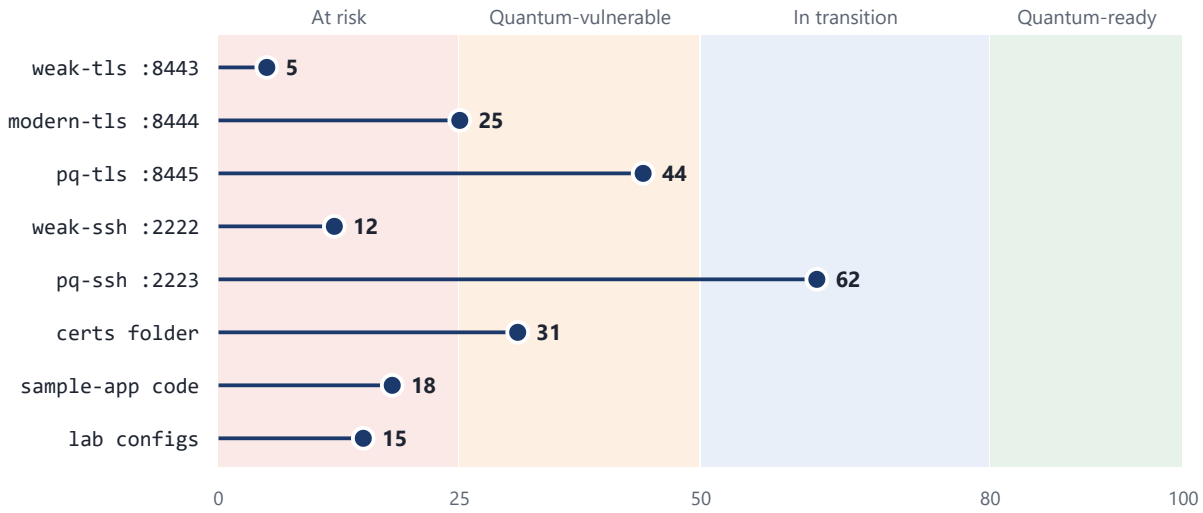


Figure 5 · The same scores placed on the four grade bands.

How to read it

- **weak-tls (5)** and **weak-ssh (12)** have Critical findings (TLS 1.0, RSA-1024, `ssh-rsa` …), so they are capped and land in “At risk”.
- **modern-tls (25)** is a textbook classical server: nothing broken today, but every public-key part is quantum-vulnerable.
- **pq-tls (44)** offers `X25519MLKEM768`, `SecP384r1MLKEM1024` and `MLKEM1024`, but its *certificate* is still ECDSA. The key-exchange category scores 81, while the certificate key and signature score 25 each, giving an average of 44. That’s honest: a quantum computer couldn’t read traffic from clients that use the hybrid groups, but it could forge the server’s certificate. (Old clients falling back to `x25519` stay exposed too.)
- **pq-ssh (62)** uses only hybrid key exchange but an Ed25519 host key, so it’s “In transition”.
- The **code** and **config** targets each contain many deliberately bad examples, so they score low too.

> **TRY IT**
>
> Start the lab (`docker compose -f lab/docker-compose.yml up -d --build`), scan only `localhost:8445` with `--html pq.html`, and find in the report which two findings hold its score down.

PART 2

How it works inside

The code base module by module: how the scanners talk to servers at the byte level, how findings are rated and scored, how reports are produced safely, and how all of it is tested. Keep the source open next to this part.

8. Map of the code

About 4,000 lines of Python in `src/qrisk/`. The structure follows the big picture: an entry point, an orchestrator, one scanner per layer, a shared rules engine and data model, and one module per output format.

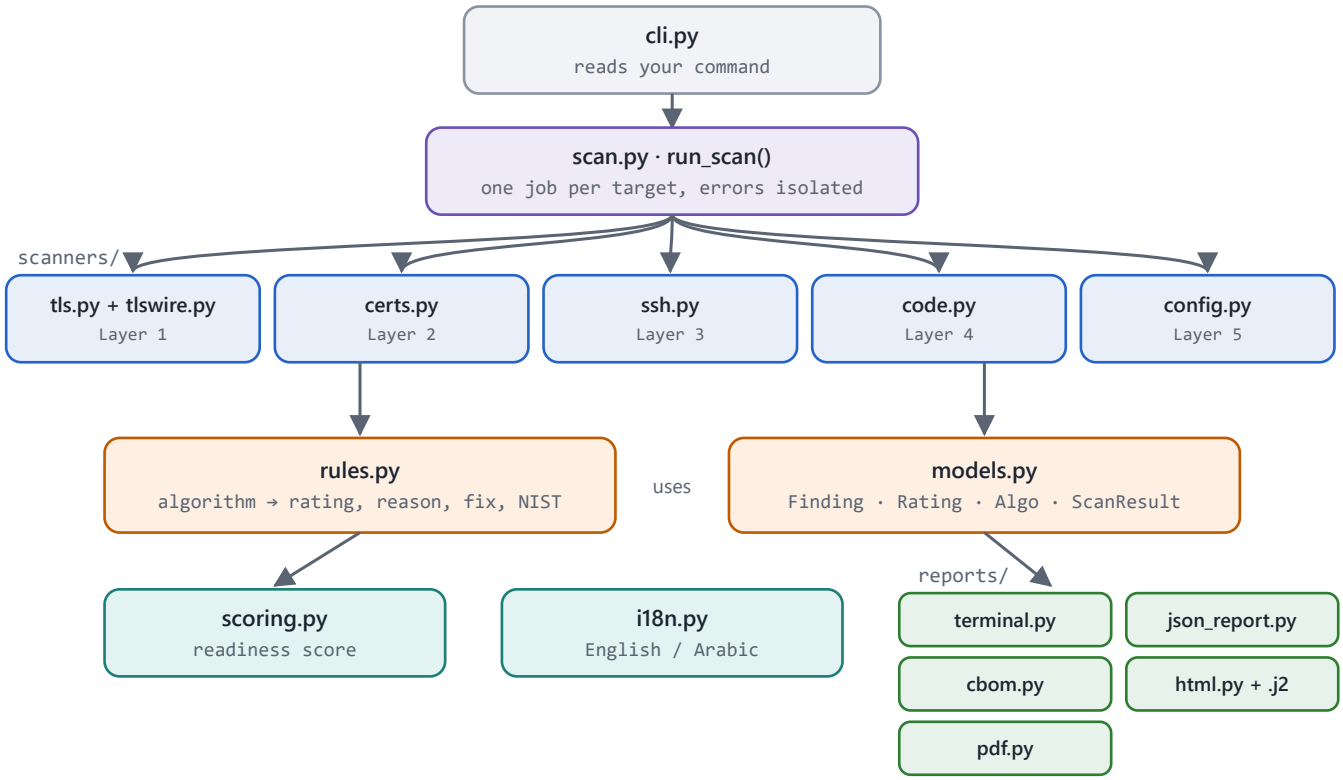


Figure 6 · Module map. Arrows show who calls whom.

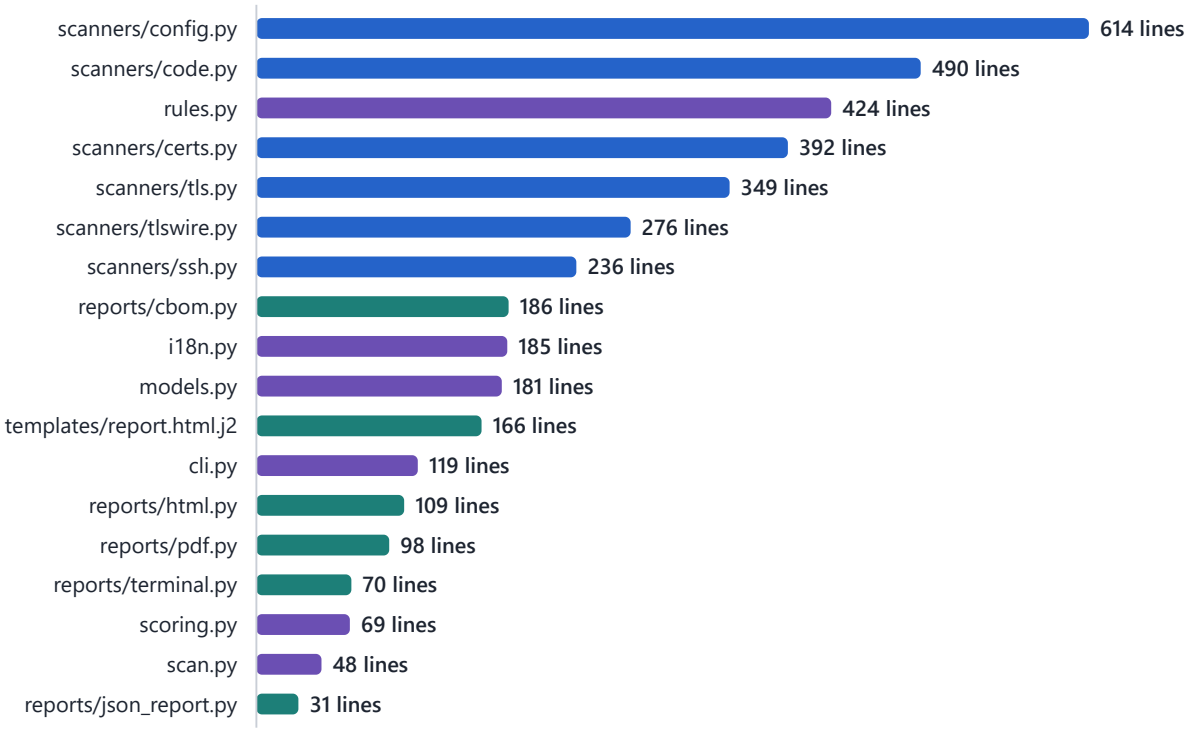


Figure 7 · Lines per module. Blue = scanners, teal = reports, purple = core.

> **SUGGESTED READING ORDER**
>
> `models.py` → `rules.py` → `scoring.py` → `scan.py` → `cli.py` → one scanner (start with `ssh.py`, the simplest live scanner) → `tlswire.py` → `tls.py` → `reports/`.

9. The data model and the life of a finding

Everything in QRisk passes through a handful of small classes in `models.py`:

CLASS	WHAT IT HOLDS
`Rating`	The four ratings, plus their label, meaning, points and sort order.
`Algo`	An algorithm described the way the CycloneDX CBOM wants it: name, primitive (*kem*, *signature*, *ae*…), parameter set, curve, mode, classical security bits, NIST quantum security level.
`CertInfo`	Facts about one X.509 certificate: subject, issuer, validity dates, SHA-256 fingerprint, key and signature algorithms.
`Finding`	One rated piece of crypto: target, layer, category, asset, rating, reason, recommendation, NIST deadline, algorithm, location, certificate, free-form details.
`Note`	Something worth telling the user that isn't rated: info, warning or error (e.g. "Could not connect").
`ScanResult`	All findings and notes from a run, plus the list of targets and timestamps.

The rules engine adds one more: a `Verdict` (rating + reason + recommendation + NIST deadline + `Algo`). A scanner turns a verdict into a `Finding` by adding *where* it was found.



Figure 8 · The life of a finding: a TLS server picks group `0x11EC`; the rules engine names it `X25519MLKEM768` and rates it Hybrid; the scanner wraps the verdict in a Finding; scoring and every report format consume it.

> **DESIGN IDEA: ONE SOURCE OF TRUTH**
>
> Scanners never decide ratings themselves. They describe what they saw and ask `rules.py`. So the live SSH scan and the `sshd_config` audit rate `hmac-md5` identically, because both call `rate_ssh_mac()`. A test checks they agree.

10. The rules engine

`rules.py` is the project's knowledge base. Each function takes an algorithm name or parameters and returns a `Verdict`:

FUNCTION	RATES
`rate_public_key(kind, bits, curve)`	RSA, EC, Ed25519/448, X25519/448, DSA, DH: by size and curve
`rate_pqc(name, level, …)`	ML-KEM, ML-DSA, SLH-DSA (by OID or name)
`rate_signature(name, hash, kind)`	Certificate signatures (MD5/SHA-1 → Critical)
`rate_tls_version(code)`	SSL 3.0 … TLS 1.3
`rate_tls_group(code)`	TLS key-exchange groups by their 2-byte code (`0x11EC` = X25519MLKEM768)
`rate_tls12_kex(kex)`	ECDHE / DHE / static RSA / anonymous
`rate_bulk_cipher(text)`	AES/ChaCha/3DES/RC4/NULL/EXPORT/Camellia… from a suite or cipher name
`rate_ssh_kex / _hostkey / _cipher / _mac`	SSH algorithms by name

```
# rules.py, rate_public_key("RSA", bits): the size picks the rating and the deadline
bits < 2048   → Critical             already disallowed
bits == 2048  → Quantum-vulnerable   NIST IR 8547: deprecated after 2030, disallowed after 2035
bits > 2048   → Quantum-vulnerable   NIST IR 8547: disallowed after 2035
```

11. Layer 1: TLS servers

`scanners/tls.py` (the logic) and `scanners/tlswire.py` (raw TLS bytes) run six steps against one host and port:

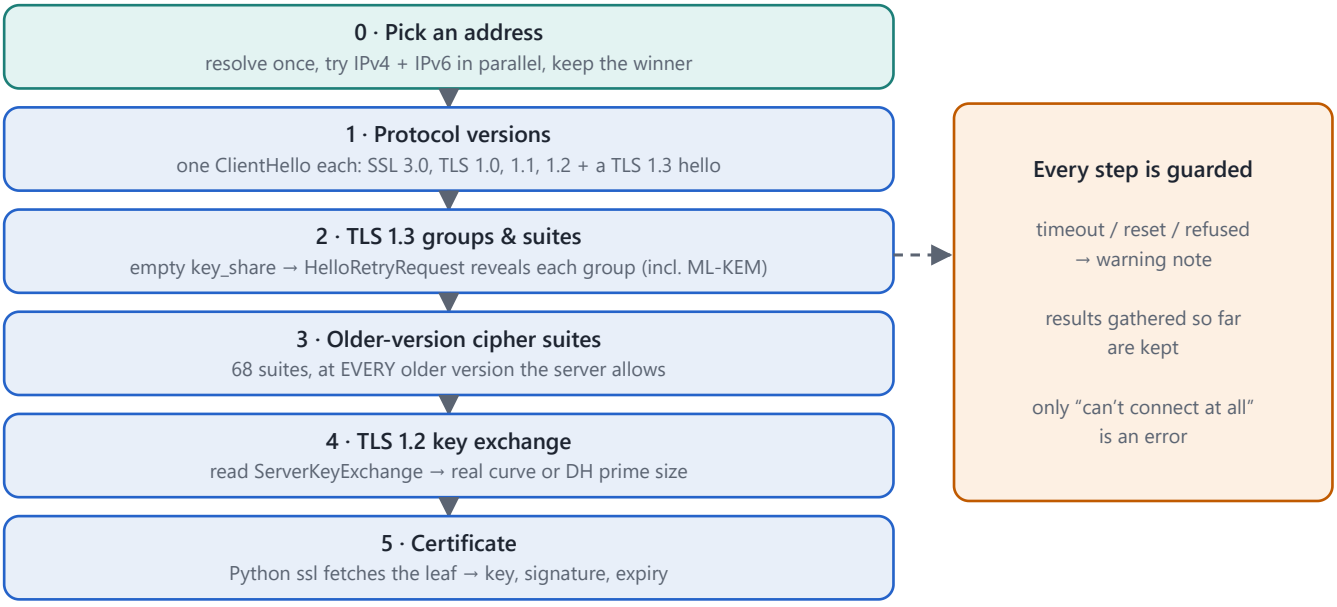


Figure 9 · The TLS scan. Steps 2–5 are each wrapped in a guard, so a server that starts resetting connections halfway still leaves useful results.

Why build TLS messages by hand?

Python's built-in `ssl` module uses the machine's OpenSSL, which may be too old to speak ML-KEM, and it only tells you what was *negotiated*, not everything the server *accepts*. So `tlswire.py` writes ClientHello messages byte by byte and reads only the first reply (the ServerHello). That is enough to learn the server's choice, without finishing a handshake.

```
build_client_hello(sni, version, cipher_suites, groups, tls13=False)
  record header   16 03 01 LL LL
  handshake       01 LL LL LL   (ClientHello)
  body            version · 32 random bytes · session id · cipher suites · compression
  extensions      server_name · supported_groups · signature_algorithms · …
                  + (TLS 1.3) supported_versions = [0x0304] · key_share = EMPTY
```

The HelloRetryRequest trick (how ML-KEM is detected)

In TLS 1.3 the client normally sends a *key share*: a public key for one group. QRisk deliberately sends **none**. A server that supports one of the listed groups must answer with a **HelloRetryRequest** naming the group it wants. QRisk recognises it by a fixed "random" value (`CF21AD74…8C5E079E09E2C8A8339C`) and reads the chosen group from its key_share extension.

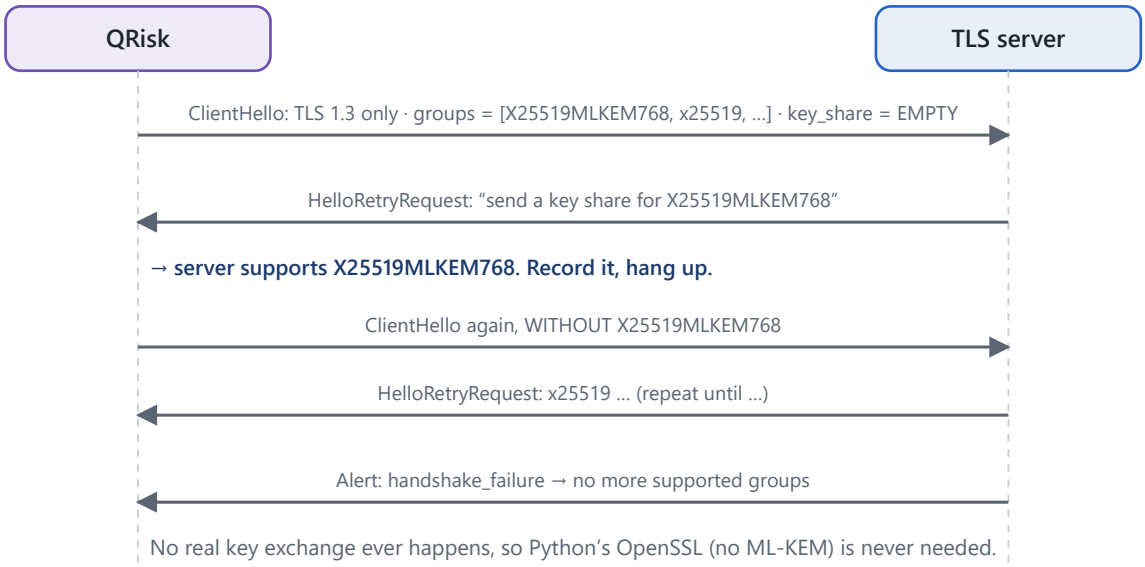


Figure 10 · Learning which groups a server supports without ever doing a real key exchange.

The offer-and-remove loop

A server answers with only *one* choice per connection. To list everything it supports, QRisk offers a list, records the choice, removes it, and asks again. The function is `_enumerate()` in `tls.py`.

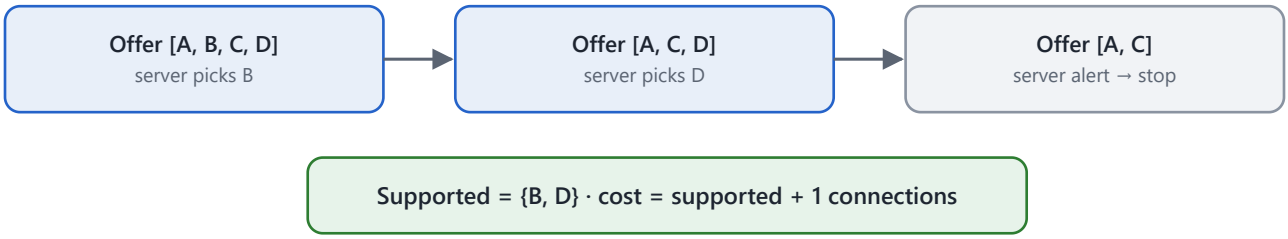


Figure 11 · Enumerating supported options costs one connection per supported item, plus one.

Older versions, cipher suites and the TLS 1.2 key exchange

- **68 TLS 1.2-era suites** are probed, including the dangerous ones (NULL, EXPORT, DES, RC4, anonymous) and the uncommon ones (Camellia, ARIA, SEED, CCM). The highest older version is enumerated in full; **each lower version** only looks for suites not already found. That's how "3DES only on TLS 1.0" is caught: it's reported as `TLS_RSA_WITH_3DES_EDE_CBC_SHA (TLS 1.0)`.
- In TLS 1.2 the **ServerKeyExchange** message isn't encrypted. QRisk reads the actual curve (e.g. `x25519`) for ECDHE suites, and the size of the prime for DHE suites. A 1024-bit DHE prime is flagged Critical (the Logjam attack).
- **Certificate:** Python's `ssl` fetches the leaf certificate without verifying it, then a second connection checks whether this machine trusts it. `certs.analyze_certificate()` rates the key, the signature and the expiry.

12. Layer 2: certificate and key files

`scanners/certs.py` walks a folder (skipping `.git`, virtual environments, `node_modules` …) and reads every file with a known extension. It recognises the format by content, not just by name:

FORMAT	HOW IT'S READ
PEM (`-----BEGIN …-----`)	Each block separately: certificates, CSRs, private/public keys, OpenSSH keys, DH parameters. Encrypted keys are noted, not opened.
DER	Tried as certificate, then private key, then public key.
PKCS#12 (`.p12` `.pfx`)	Opened with no password, an empty password, or `changeit`. Opening with a default password is itself reported.
JKS / JCEKS (`.jks` …)	Parsed by hand: certificates in these formats are **not encrypted**, so they can be read without the password. The keystore format itself is rated Critical (weak key protection).
OpenSSH `.pub`	`ssh-ed25519 AAAA…` lines.

POST-QUANTUM KEYS BY THEIR OID

The `cryptography` library can't load ML-DSA or ML-KEM keys yet. So when loading fails, QRisk looks for the algorithm's **object identifier** inside the file: `2.16.840.1.101.3.4.3.18` means ML-DSA-65. That's how the lab's ML-DSA key and certificate are rated Quantum-safe. (This gap was found by running the lab; see NOTES.md #1.)

13. Layer 3: SSH servers

SSH negotiates algorithms **in clear text** before any encryption starts. `scanners/ssh.py` simply reads that list.



Figure 12 · Banner, KEXINIT (rated), and the optional second connection that reveals the RSA host key size.

- **KEXINIT** contains ten comma-separated name lists. QRisk rates key exchange, host-key algorithms, ciphers and MACs, and skips protocol markers such as `ext-info-s` and `kex-strict-s-v00@openssh.com`.
- Host-key *algorithm names* don't say how big an RSA key is, so QRisk starts a real key exchange, receives the server's public host key in `KEX_ECDH_REPLY`, reads the modulus size, and disconnects. Nothing is authenticated and no session exists.

14. Layer 4: source code

`scanners/code.py` finds crypto API calls in Python, Java/Kotlin, JavaScript/TypeScript and Go, and reports the file and line.

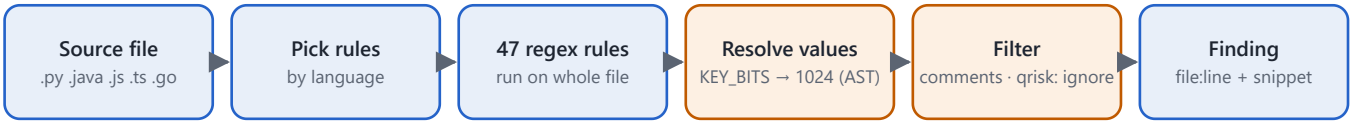


Figure 13 · Each rule is a regular expression plus a small handler that turns a match into a rating.

```
# one of the 47 rules: Java key-pair generation. The handler looks ahead for .initialize(size)
_r("java-keypair", JAVA, r'\b(KeyPairGenerator|KeyAgreement|KeyFactory)\.getInstance\(\s*"([\w-]+)"',
…)

KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");   →  RSA-2048, Quantum-vulnerable,
Main.java:1
kpg.initialize(2048);
```

- **Whole-file matching:** patterns run on the whole file, so a call split over several lines still matches and its arguments can be read.
- **Constant resolution:** real code writes `key_size=KEY_BITS`, not `2048`. For Python, QRisk parses the file with Python's own `ast` module and evaluates constants, simple arithmetic (`512 * 2`), default arguments and class attributes. For other languages, a regex looks for `KEY_SIZE = 1024`.
- **Fewer false positives:** it matches *configuration*, not mentions. For example `= ssl.CERT_NONE` is flagged but `== ssl.CERT_NONE` isn't. A private key needs real base64 after its header. Python's `usedforsecurity=False` is respected. `qrisk: ignore` silences a line.

> **KNOWN LIMIT**
>
> Only lines that *start* with a comment are skipped, so matches inside block comments, docstrings or strings still count. Values are resolved within one file, not across files.

15. Layer 5: configuration files

`scanners/config.py` (the largest module) recognises nginx, Apache, `openssl.cnf` and OpenSSH files by their contents, parses them into directives with line numbers, and analyses them.

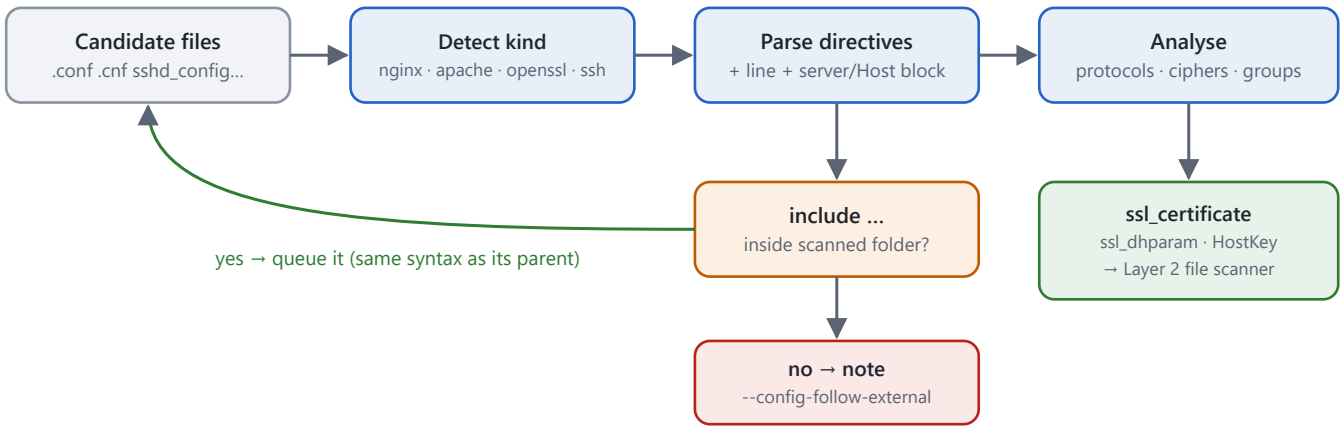


Figure 14 · Includes are followed (in the parent's syntax) only while they stay inside the scanned folder. Referenced certificates and keys go to the Layer 2 scanner.

Things the config scanner catches that are easy to miss

CONFIG	WHY IT MATTERS
`SSLProtocol all -SSLv3` (Apache)	Looks safe, but still enables TLS 1.0 and 1.1. QRisk resolves the `+` / `-` / `all` logic.
`CipherString = DEFAULT@SECLEVEL=1`	Quietly allows RSA-1024 and SHA-1. QRisk splits off the `@SECLEVEL` .
`ssl_ciphers 'HIGH:AES256-SHA'`	`AES256-SHA` is a static-RSA suite (no forward secrecy).
`HostKeyAlgorithms -ssh-rsa`	The `-` *removes* an algorithm, so it is correctly **not** flagged.
no `ssl_ecdh_curve` at all	A note explains the default: OpenSSL 3.5+ offers X25519MLKEM768 on its own; older versions offer no post-quantum group.

Each finding records its **block**: `server shop.example.com` (nginx), `VirtualHost *:443` (Apache) or `Host legacy.example.com` (SSH). It's shown next to `file:line` in every report.

16. Outputs

FORMAT	MODULE	NOTES
Terminal	`reports/terminal.py`	Rich colour table sorted worst-first, top recommendations, notes.
JSON	`reports/json_report.py`	Every finding and note as plain data for other tools.
CBOM	`reports/cbom.py`	CycloneDX 1.6 *Cryptographic Bill of Materials*, validated against the official schema in the tests.
HTML	`reports/html.py` + template	Executive summary (score gauge, NIST deadlines, priority actions), then technical detail.
PDF	`reports/pdf.py`	Prints the HTML with an installed Chrome/Edge/Chromium in headless mode.
Arabic	`i18n.py`	`--lang ar` : right-to-left layout; executive part and recommendations translated.

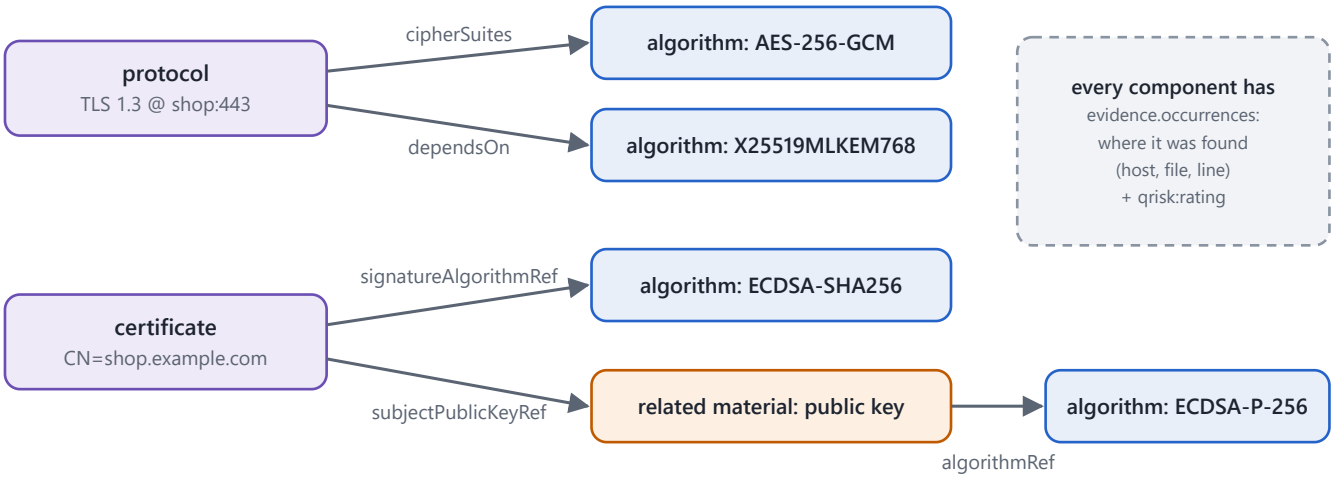


Figure 15 · Inside the CBOM: protocols, certificates and key material point to algorithm components, and every component records where it was found.

Three details worth understanding in the reports

- **Escaping (security).** The report shows text that comes from scanned systems, such as certificate names and SSH banners. Jinja2 runs with `autoescape=True`, so `<script>` becomes harmless text. Only the report's own `code` filter produces markup, and it escapes its input first.
- **Arabic text direction.** In right-to-left text, the Unicode bidirectional algorithm reorders a Latin date like `2026-11-06` into `06-11-2026`. QRisk wraps dates and version numbers in invisible isolate characters (LRI … PDI), and renders commands as `<code dir="ltr">`.
- **PDF printing.** QRisk uses a temporary browser profile, waits until the PDF file exists *and* its size stops changing (Edge 157+ returns early), and uses a light print theme even when the computer is in dark mode.

17. Robustness and security by design

A security tool is run against hostile or flaky systems, and its output is trusted by people who make decisions. Several design choices protect both sides:

RISK	DEFENCE
A server times out after 40 handshakes	`run_scan()` isolates each target; inside a TLS scan each step is guarded and partial results are kept with a warning.
A malicious certificate name contains a script	All report text is escaped (see above); a test uses real hostile certificates.
Scanning a client's copied config pulls in your own `/etc` files	Includes and referenced files outside the scanned folder are skipped unless `--config-follow-external` is given.
CI shows green when nothing was scanned	`--fail-under` fails on errors and empty scans (Figure 16).
Slow or broken IPv6 path	Happy Eyeballs: pick a working address once per scan (Figure 17).

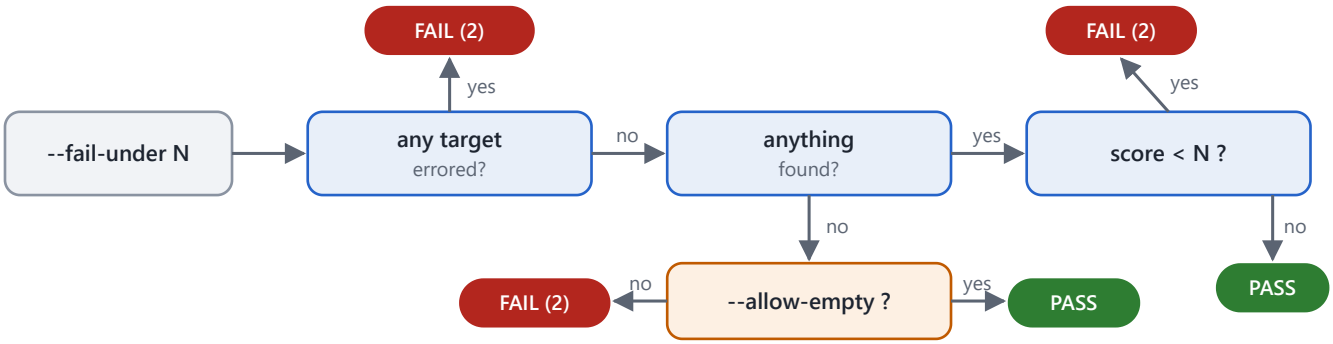


Figure 16 · How `--fail-under` decides. "No data" must never look like "good data".

Happy Eyeballs: a performance bug in detail

On Windows, `localhost` resolves to IPv6 `::1` first, but Docker published the lab on IPv4 only. Every one of the ~40 connections in a TLS scan tried IPv6, waited about 2 seconds for it to fail, then used IPv4. The fix (`tlswire.pick_address()`) resolves the name once and tries all addresses in parallel, as browsers do (RFC 8305).

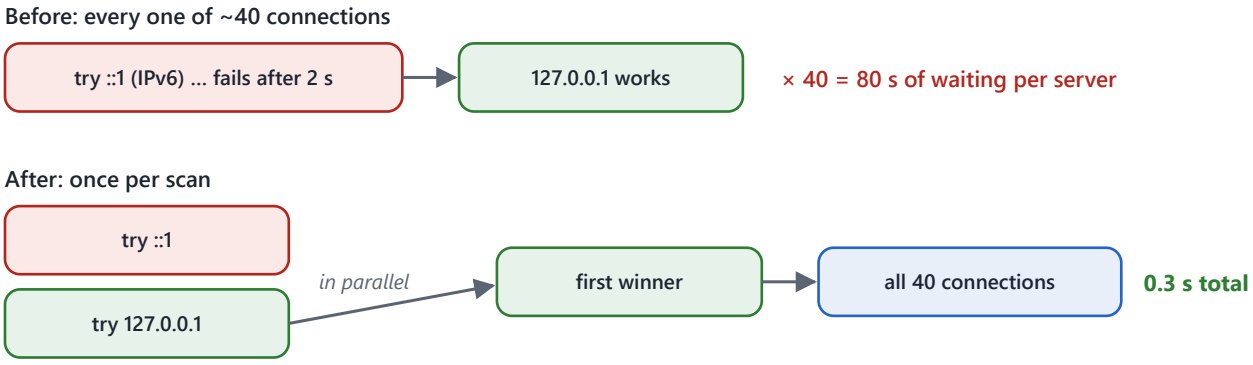


Figure 17 · Before and after: the same work, without 40 failed IPv6 attempts.

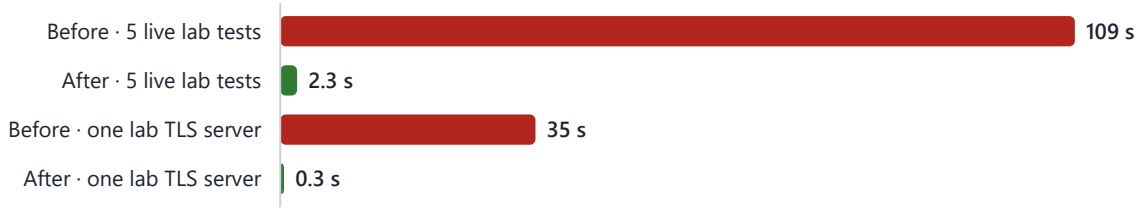


Figure 18 · Measured times. The lab TLS server time before the fix is the middle of a 30–40 s range.

18. Testing and the Docker lab

97 tests run in about 15 seconds. Most use small servers that run inside the test itself: a real TLS server from Python's `ssl` module, a fake SSH server that speaks just enough protocol, and a fake legacy TLS server that accepts RC4 only on TLS 1.0. So the tests need no internet access.

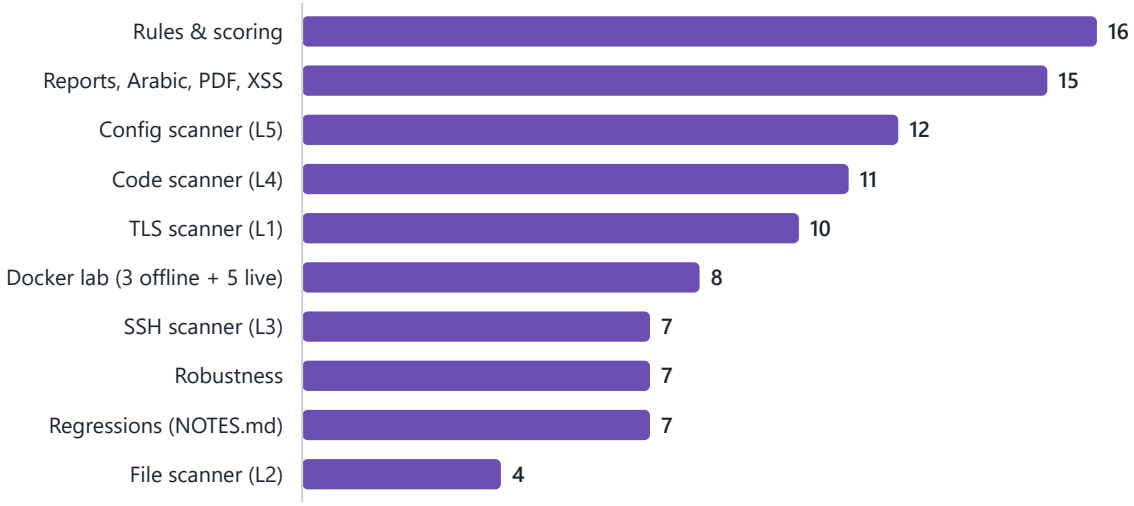


Figure 19 · Tests per area. The 5 live lab tests skip (with instructions) when the lab isn't running.

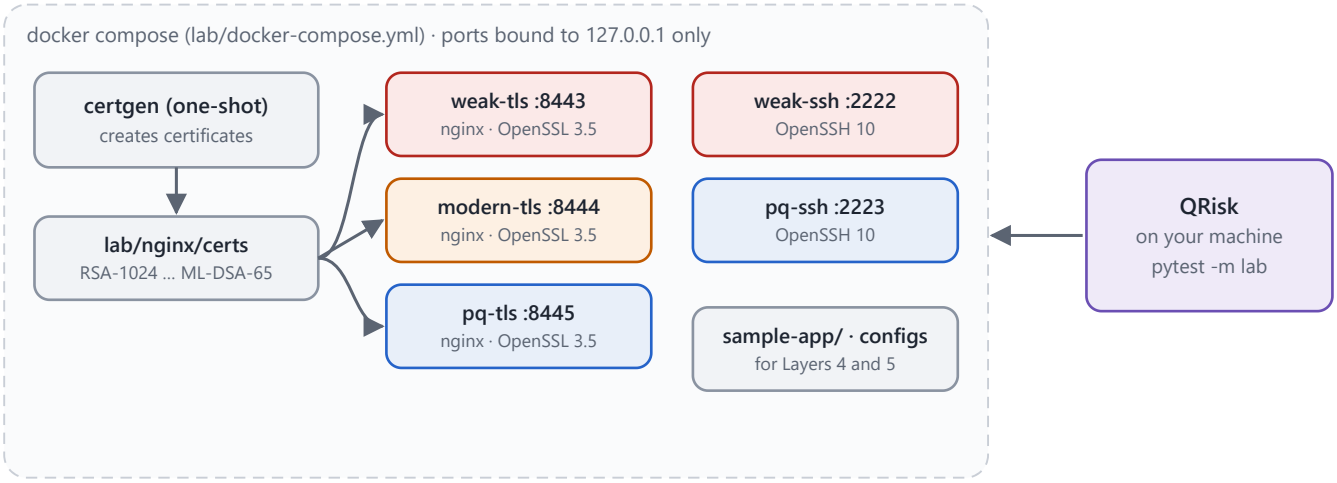


Figure 20 · The Docker lab: Alpine 3.22 images with OpenSSL 3.5 and OpenSSH 10, so post-quantum support is built in.

Kinds of tests and what they protect

KIND	EXAMPLE
Unit tests of the rules	`rate_public_key("RSA", 1024)` is Critical; no Critical verdict carries a future deadline.
Wire-format tests	A hand-built HelloRetryRequest parses to group `0x11EC`.
Scanner tests with local servers	A real TLS server shows TLS 1.2 + 1.3, `x25519`, RSA-2048.
Standards tests	The CBOM validates against the official CycloneDX 1.6 schema.
Security tests	A certificate named `<script>…` appears escaped in both languages.
Regression tests	One per bug in NOTES.md. Some re-create the bug to prove the test can fail.
Behaviour-level tests	The Arabic date is checked with the same bidi algorithm browsers use, not by looking for a marker character.
Live lab tests	Each container gets the rating it was built for.

Continuous integration

`.github/workflows/ci.yml` has two jobs. **test** runs the offline suite on Ubuntu and Windows with Python 3.10–3.13, then scans QRisk's own code with `--fail-under 80 --allow-empty`. **lab** starts the Docker lab, runs the live tests with `QRISK_LAB_REQUIRED=1` (so a lab that fails to start fails the build), and uploads an HTML report and CBOM.

> **WHERE TO LEARN FROM MISTAKES**
>
> `NOTES.md` records 22 bugs as problem → cause → fix → lesson, including the report XSS, the `p += u4()` evaluation-order bug, the backwards Arabic date and the IPv6 slowdown, each mapped to its regression test.

19. Glossary

TLS
: The protocol behind HTTPS. 1.3 is current; 1.0/1.1 are obsolete.

Handshake
: The opening messages where client and server agree on algorithms and keys.

ClientHello / ServerHello
: The first message from each side; they list and pick algorithms.

HelloRetryRequest (HRR)
: A TLS 1.3 server's "please resend with a key share for this group".

ML-KEM (FIPS 203)
: NIST's post-quantum key encapsulation (formerly Kyber).

ML-DSA (FIPS 204)
: NIST's post-quantum signature (formerly Dilithium).

SLH-DSA (FIPS 205)
: Hash-based post-quantum signature (formerly SPHINCS+).

Hybrid
: Classical + post-quantum combined; safe if either holds.

Key exchange (KEX)
: How two parties agree on a secret key in public. The part quantum computers break.

Group
: A named key-exchange method in TLS 1.3, e.g. `x25519` or `X25519MLKEM768`.

Cipher suite
: A bundle of algorithms used together, e.g. `TLS_AES_256_GCM_SHA384`.

Forward secrecy
: Stealing the server's long-term key later doesn't decrypt old traffic. Static RSA lacks it.

SNI
: Server Name Indication: the hostname the client asks for, sent in the ClientHello.

ServerKeyExchange
: TLS 1.2 message carrying the server's ephemeral key; it names the curve.

KEXINIT
: SSH's clear-text list of supported algorithms.

Host key
: An SSH server's long-term identity key.

Shor / Grover
: Quantum algorithms: Shor breaks public-key crypto, Grover weakens brute force.

OID
: Object identifier: a dotted number naming an algorithm inside certificates and keys.

PEM / DER / PKCS#12 / JKS
: File formats for certificates and keys.

CBOM
: Cryptographic Bill of Materials: a standard, machine-readable crypto inventory (CycloneDX 1.6).

Logjam / Sweet32
: Real attacks on 1024-bit DH and on 64-bit block ciphers like 3DES.

XSS
: Cross-site scripting: injected HTML/script running in someone's browser.

Happy Eyeballs
: Trying IPv6 and IPv4 at once and using whichever connects first (RFC 8305).

Bidi
: The Unicode algorithm that orders mixed left-to-right and right-to-left text.

20. A study plan with exercises

Work through these in order. Each step pairs reading with something to run. All commands run from the `qrisk-scanner` folder.

#	READ	DO
1	Part 1 of this guide	`qrisk scan --tls github.com` and explain each Critical and Quantum-vulnerable line.
2	`models.py` , `rules.py`	In a Python shell: `from qrisk.rules import *; rate_public_key("RSA", 3072)` . Predict the deadline before running it.
3	`scoring.py` , `tests/test_scoring.py`	Work out by hand why pq-tls scores 44 (section 7), then check with the lab.
4	`ssh.py`	`qrisk scan --ssh github.com` . Find where `ext-info-s` is filtered out and why.
5	`tlswire.py` , then `tls.py`	Add a `print()` in `_enumerate()` and watch the offer-and-remove loop against `pq.cloudflareresearch.com` .
6	`certs.py`	Make a test certificate with OpenSSL and scan it with `--path` . Then make one with `-sha1` and compare.
7	`code.py`	Write a small Python file using `KEY_BITS = 1024` and `rsa.generate_private_key(65537, KEY_BITS)` ; scan it with `--code` .
8	`config.py`	Scan `lab/` with `--config` and find the 3DES setting that the live scan never sees.
9	`reports/` , `i18n.py`	Produce `--html` and `--lang ar --pdf` reports of the lab and compare them.
10	`NOTES.md` , `tests/`	Pick one bug, delete its fix, and run `pytest` : watch its regression test fail. Then restore it.

SELF-CHECK QUESTIONS

1. Why is AES-256 rated Quantum-safe while ECDSA P-384 is not?
2. How can QRisk learn that a server supports ML-KEM when its own OpenSSL can’t speak ML-KEM?
3. Why does a classical TLS 1.3 server score 25 and not something higher?
4. What would go wrong if the report used `select_autoescape()` again?
5. Why does `--fail-under` fail when nothing was found, and when would you pass `--allow-empty` ?
6. Why is `TLS_RSA_WITH_AES_256_GCM_SHA384` rated Critical when AES-256-GCM is strong?

Answers: §1 (Grover vs Shor) · §11 (HelloRetryRequest) · §5 (only public-key categories count) · §16 (XSS from scanned text) · §17 (no data ≠ good data) · §10/§11 (static RSA key exchange has no forward secrecy).